COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate route for games

04

Data structures and their costs

Module 13 / expanded Volume 03

Four lessons, worked examples, full practical requirements and 20 lesson questions.

PACKAGE 04 / EXPANDED VOLUME 03 / SEPTEMBER 28, 2026
Independent study, implementation, evidence and review.

Contents

13.1 — Complexity, arrays and linked structures 3

13.2 — Stacks, queues and ring buffers 6

13.3 — Hash tables and collision policies 9

13.4 — Trees, heaps and priority queues 12

Continuity with the original program 15

Primary reference directory 15

Reading by topic 17

Module 13 — Data structures and their costs

Prerequisites: Modules 06 and 11-12.

Project: Implement educational versions of a dynamic array, queue, hash table and heap, each with invariants and tests. Compare them to standard containers without replacing standard code in production merely for practice.

Primary references: R13, R06. Four lessons,20 expanded quiz questions and a module project that contributes to GateC.

13.1 — Complexity, arrays and linked structures

Learning outcome: Predict asymptotic work while recognizing locality and constants.

Define the cost model before using Big O

Asymptotic analysis describes how an operation count grows with input size under a chosen model. O(n) is an upper-bound class, not an exact runtime or a claim that all linear algorithms are equally fast. When you can justify both upper and lower growth bounds, Theta notation expresses the tighter classification. Separate worst-case, average-case and amortized claims.

For a contiguous array, indexing is constant-time under the ordinary random-access model, scanning n elements is linear, and inserting at the beginning requires shifting elements. For a linked list, insertion at an already-known position can be constant-time, but finding that position may be linear. Include the lookup cost in the actual workload.

A game scheduler that repeatedly searches a linked list for an insertion point does not get constant-time insertion overall merely because the final pointer rewiring is short. Write the complete operation: locate, validate, allocate, mutate and update any indices. The model should include the dominant work you are trying to compare. [R13: analysis, arrays; R06: sums]

Why geometric growth is amortized constant time

A growable array keeps size <= capacity. When full, it allocates more storage, copies or moves the existing elements and then appends. If capacity increases one slot at a time, appending n elements copies 0+1+…+(n-1) = n(n-1)/2 old elements. That is quadratic total copy work.

If capacity doubles from 1 to 2 to 4 and so on, the copy counts form a geometric series. Before n appends, the total number of relocated old elements is less than 2n, with details depending on initial capacity. Add the n new writes and the total remains linear. Therefore the amortized cost per append is constant under this model, even though a particular growth append costs linear time.

Amortized analysis does not assume random input. It spreads the expensive operations across the sequence. It also does not guarantee a short latency for every frame. A real-time path might reserve a known bound before gameplay or use another allocation strategy to avoid a growth spike.

A deliberately restricted educational buffer

The lab's growable buffer stores integers, not arbitrary C++ objects. It uses an owning array and tracks size, capacity and relocation count. Restricting the element type lets you focus on the size/capacity invariant before learning raw storage, placement construction, partial element construction and allocator traits.

Before expanding capacity, check that the growth calculation and byte count remain within the chosen teaching limit. Allocate the candidate array first, copy existing integers, then commit the new storage. A failed allocation must not lose the original owner. This is the same prepare-then-commit pattern introduced in resource management.

Bounds-checked access rejects an index equal to size, even if that index is below capacity. Allocated storage is not the same as a valid logical element. Test empty access, every growth boundary and value preservation after relocation. Do not advertise this restricted buffer as a production replacement for `std::vector`.

Locality explains why equal asymptotic work can differ

A contiguous scan touches nearby elements. A linked traversal follows pointers whose targets may be scattered. Both scans are O(n), but their constants and interaction with the memory hierarchy can differ substantially. That is a hypothesis to measure on the real workload, not a universal numerical claim.

Collect relocation counts separately from elapsed time. Counts can validate the derived model exactly. Timings also reflect allocation, cache behavior, compiler decisions, clock noise and other activity. A timing curve alone does not prove the asymptotic bound, especially over a narrow range of small inputs.

For the module project, compare your educational structures with standard containers using the same logical operation trace. Keep the custom implementation because it exposes invariants and costs, not because writing it yourself makes it automatically faster or safer. [R15; R13]

Predict eight appends

With exact-fit growth, capacities are 1,2,3,4,5,6,7,8 and relocation count is 0+1+2+3+4+5+6+7 = 28. With doubling, capacities reach 1,2,4,8 and relocation count is 0+1+2+4 = 7. Both store the same eight final integers.

At the ninth append, doubling relocates eight elements in one operation. This is consistent with amortized constant time: an individual append can still be expensive. The exact-fit strategy relocates eight as well, but has already paid much more over the preceding sequence.

Fill a table with size, capacity, relocation count for each append before running. Compare the counters exactly; compare elapsed time only after both versions preserve all values.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_13_1.cpp`. Pseudocode excerpts are not standalone C++ source.

```
// Cost model for exact-fit growth over n appends:
// moved = 0 + 1 + ... + (n - 1)
//       = n * (n - 1) / 2
// Compute this formula only after checking its numeric range.
// Doubling copies 1 + 2 + 4 + ... < 2*n old elements.
```

Common failure modes

O(1) insertion at a known linked node does not include finding the node. Amortized constant time does not bound worst-case frame latency. Size is the access boundary; spare capacity is not permission to read an element.

Practical assignment

Original requirement: Implement a growable integer buffer for study. Count copied elements under doubling and one-at-a-time growth. Pass: derive the growth behavior and separately measure elapsed time without equating it to the proof.

1. Implement an integer-only buffer with size/capacity invariants and two growth policies.
2. Instrument relocated-element counts and predict the sequence for n = 8 and n = 9.
3. Test value preservation, empty access, exact growth boundaries and a chosen capacity budget.
4. Compare counts analytically, then collect separate Release timings against std::vector.

Required cases: empty buffer has size 0; out-of-range access rejected; n=8 exact-fit moves 28; doubling moves 7; n=9 doubling moves 15; values preserved through growth; capacity limit checked before arithmetic.

Evidence to submit: Growth trace, geometric-series argument, invariant tests and timing results separated from the proof.

Independent transfer: A game must never allocate during a frame. Explain why amortized O(1) is insufficient and design a capacity-planning policy with an explicit failure result.

Assigned primary reading: R13: arrays, cost models and asymptotic analysis; R06: sums.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Does O(n) specify an exact runtime?
2. What is the total 1+...+(n-1) proportional to?
3. How does amortized analysis differ from random-input average-case analysis?
4. Can an amortized O(1) append take O(n) time on one call?
5. Why does timing two small input sizes not prove an O(n log n) bound?

13.2 — Stacks, queues and ring buffers

Learning outcome: Match ordering semantics to the intended workflow.

Ordering is a semantic requirement

A stack removes the most recently inserted item first: last in, first out. A queue removes the oldest inserted item first: first in, first out. A priority queue removes according to priority rather than arrival order. Choose from the meaning of the work: undo history, event ingestion and pathfinding frontiers need different rules.

A queue that drops or reorders commands changes simulation behavior. Specify whether capacity is bounded, whether insertion can fail, and what happens when the structure is empty. Do not let overwriting an unread command become an accidental overflow policy.

This lesson's queue is bounded and single-threaded. Push returns false when full; pop returns an optional integer when nonempty. The caller decides whether to retry, reject the command or apply a documented higher-level policy. The data structure does not silently choose gameplay behavior. [R13: elementary structures]

A ring buffer reuses a fixed allocation

Store a fixed-capacity array, a head index naming the next readable slot, and a count of live entries. The next write index is `(head + count) % capacity`. A push writes there and increments count; a pop reads at head, advances head modulo capacity and decrements count.

Count distinguishes empty from full: count = 0 means empty and count = capacity means full. Another valid design reserves one slot so equal read and write indices always mean empty. Do not mix the two conventions. The usable capacity and full test would disagree.

Reject zero capacity or use a representation whose zero-capacity case avoids modulo by zero. The included compile-time-capacity ring requires capacity > 0. Capacity one is a valuable boundary test because every successful operation wraps the index.

State the invariant and trace the transitions

For the count-based design, require 0 <= count <= capacity and head < capacity. Exactly count logical entries are readable in FIFO order beginning at head. Slots outside that logical range do not contain live queue entries, even if they contain old integer bits from earlier operations.

A failed push must leave head, count and existing contents unchanged. A failed pop must not read a slot. A successful pop returns the old head value before updating the indices. These statements are short enough to inspect in every branch of the implementation.

Trace capacity three: push A, B, C; reject D; pop A; push D; pop B, C, D. The physical order in the backing array can be D, B, C while the logical order is B, C, D. Tests must verify logical order, not assume that array index zero is always the front.

Single-thread correctness is not concurrency correctness

A ring buffer with correct sequential transitions can still have data races when a producer and consumer access it concurrently. Atomic indices, ordering of payload publication, ownership of each slot and wraparound reasoning become separate requirements. A random stress test is not a substitute for a memory-order argument.

Do not add a few atomics and label this exercise lock-free. The current module deliberately stays single-threaded. Later operating-system and concurrency lessons will revisit the queue with the required memory-model background.

For this lab, compare thousands of deterministic push/pop operations against `std::deque`. After every operation compare success or absence, logical size and returned values. Include more than three times the capacity in operations, repeated empty/full transitions and capacity one. This gives much stronger evidence than one push followed by one pop.

Capacity-three trace

After push 10,20,30: head = 0 and count = 3. Another push fails with no state change. Pop returns 10; head = 1 and count = 2. Push 40 writes at `(1+2)%3 = 0`; count returns to 3. The backing array is [40,20,30], but successive pops return 20,30,40.

When the last item is popped, count becomes zero. Head may be nonzero; that does not mean the queue is nonempty. Emptiness follows the count convention. Never inspect a stale slot to infer whether an item is live.

Draw each operation as a before/after state, then run the same trace against the supplied reference.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_13_2.cpp`. Pseudocode excerpts are not standalone C++ source.

```
bool push(int value) {
    if (count_ == Capacity) return false;
    data_[(head_ + count_) % Capacity] = value;
    ++count_;
    return true;
}
```

Common failure modes

Read/write-index equality is ambiguous unless the convention resolves it. Capacity zero makes a naive modulo invalid. Old bits in an empty slot are not a queued item. Sequential tests do not establish thread safety.

Practical assignment

Original requirement: Implement a bounded queue with explicit full/empty results and wraparound tests. Pass: preserve FIFO order through more than three times its capacity in operations and never read an uninitialized slot.

1. Choose and document the count-based full/empty convention.
2. Implement bounded push/pop without dynamic allocation during operations.
3. Hand-trace wraparound at capacity three and test capacity one.
4. Differentially test a long deterministic operation sequence against std::deque and preserve the seed or trace.

Required cases: pop empty -> absent; push full -> false with unchanged state; capacity-one alternating operations; wraparound returns FIFO order; more than 3*capacity operations; no read from a logically empty slot.

Evidence to submit: State-transition table, invariant checks, failure-policy documentation and differential-test trace.

Independent transfer: An audio queue must never block but may drop old samples. Define a different overflow policy and identify which tests must change; do not silently reuse the command-queue contract.

Assigned primary reading: R13: elementary data structures; R16: later concurrency comparison.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Which item leaves first after pushing A then B onto a stack?
2. Why does a ring buffer need a full/empty convention?
3. Does a correct single-threaded queue imply a thread-safe queue?
4. After wrapping, must the front item be stored at array index zero?
5. Why must a failed push leave existing entries unchanged?

13.3 — Hash tables and collision policies

Learning outcome: Explain expected lookup behavior and its assumptions.

Hashing narrows a search; equality decides identity

A hash function maps a key into a finite hash value. Distinct keys can share the same hash. A dictionary must still compare keys using its equality rule. Equal keys must produce compatible hash values; otherwise lookup may search a different bucket from insertion.

In separate chaining, each bucket contains entries whose hash selects that bucket. Lookup computes the bucket and then checks equality among its entries. In open addressing, entries occupy a shared array and collisions follow a probing rule. Both require precise handling of empty states, existing-key updates and resizing.

For a game asset table, decide whether the same ID replaces a value, rejects a duplicate or creates a versioned record. The included educational table uses update semantics: inserting an existing key changes its value without increasing entry count. This is a contract choice, not an automatic property of hashing. [R13: hashing]

Expected constant time has assumptions

For a chaining table with n entries and m buckets, load factor alpha = n/m. Under an appropriate distribution model, expected chain length is related to alpha, so expected lookup work can remain bounded when load is controlled. A terrible hash function can put every key in one bucket and make lookup linear in n.

Hash computation and key equality have costs too. Hashing a long string is not necessarily constant time in the string length. State which input size the complexity claim concerns. Expected O(1) dictionary access usually abstracts or separately accounts for the key-processing cost.

A resizing policy keeps occupancy within a chosen range, but cannot rescue all adversarial hash distributions. The lab includes a constant-hash test specifically to expose correctness under collisions. It should remain correct even when it is deliberately slow. Do not conflate a performance failure with permission to return the wrong value.

Rehashing changes placement, not identity

When bucket count changes, recompute every entry's bucket under the new count. Copying the old bucket array without redistributing entries breaks lookup. Allocate candidate buckets, reinsert or redistribute the existing pairs, then commit the new bucket collection.

For the restricted integer table, copying into candidate buckets makes failure reasoning manageable. A general-purpose table must additionally handle throwing hashes, throwing key/value operations, iterator invalidation and allocator behavior. State these restrictions rather than claiming industrial robustness from a short classroom implementation.

The table invariant is: entry count equals the total number of entries in all buckets, keys are unique under the equality rule, and every stored key resides in its computed bucket. Verify the invariant after insert, update, erase and resize. A test that only checks size can miss a misplaced entry.

Deletion policies differ by collision strategy

In chaining, erasing one matching entry leaves the others searchable. In open addressing, marking a slot completely empty can prematurely stop a probe that should continue to a colliding key. Tombstones or a carefully justified alternative deletion algorithm preserve search continuity.

Choose chaining for the required reference implementation; explain the open-addressing deletion problem on paper. This keeps the implementation scope bounded while still teaching why collision policy affects correctness. An extension can implement tombstones and compare probe counts at different loads.

For the module comparison, feed your table and `std::unordered_map` the same deterministic operations and compare resulting key/value mappings. Do not require the same iteration order: the contract here concerns mappings, not bucket order or a vendor-specific traversal sequence.

Force three keys into one bucket

Use a hash function that always returns zero. Insert (4,40), (9,90) and (14,140). All occupy the same chain. Query 9 by comparing keys until equality succeeds. Update key 9 to 91; size remains three. Erase key 4; queries for 9 and 14 must still succeed.

After resizing, the constant hash still puts all entries in one bucket. Correctness remains intact while the worst-case search remains linear. This experiment separates collision handling from distribution quality.

For ordinary hashing with 30 entries and 10 buckets, alpha is 3, but that average alone does not tell you the maximum chain length. Inspect the distribution as well.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_13_3.cpp`. Pseudocode excerpts are not standalone C++ source.

```
for (auto& entry : buckets_[bucket_for(key)]) {
    if (entry.first == key) {
        entry.second = value; // update, not a second key
        return;
    }
}
// append a new pair only when equality found no match
```

Common failure modes

Matching hashes do not establish equal keys. Rehashing is not a raw resize of bucket storage. Erasing an open-addressed slot as empty can break later lookups. A stable logical result does not require stable hash iteration order. The educational map deliberately disables copy and move: blindly moving its bucket vector while copying its size metadata would break the moved-from invariant. A production value-like map needs a correctly specified and implemented special-member policy.

Practical assignment

Original requirement: Implement chaining or open addressing with collision tests and resizing. Pass: deliberately force many collisions, test missing keys and verify that distinct equal-hash keys remain distinct.

1. Implement a restricted chaining table with insert/update/find/erase and controlled resizing.
2. Check the full invariant after every mutation.
3. Force all keys to collide and test equal-hash distinct keys, updates and deletions.
4. Compare deterministic traces against std::unordered_map and report occupancy/probe observations separately from correctness.

Required cases: missing key -> absent; same key update does not increase size; distinct equal-hash keys preserved; erase chain head/middle/tail; all entries survive resize; negative and zero integer keys handled.

Evidence to submit: Collision-policy specification, resize reasoning, invariant checks and differential mapping tests.

Independent transfer: A mod supplies many deliberately colliding asset names. Explain why correctness tests are insufficient for a production resource-exhaustion defense.

Assigned primary reading: R13: hashing and dictionary data structures.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Does a matching hash prove two keys are equal?
2. What is the load factor for 30 entries in 10 buckets?
3. Why is expected O(1) not an unconditional guarantee?
4. Why can clearing an open-addressed slot to empty break lookup?
5. Does load factor 1 guarantee every bucket contains exactly one entry?

13.4 — Trees, heaps and priority queues

Learning outcome: Preserve structural invariants and choose the right query behavior.

Trees answer different questions

A binary search tree orders keys so a search can discard one side at each comparison. Its operation cost depends on height. An unbalanced tree can become a chain after ordered insertion, with linear height and linear search. A balanced search tree controls height to support logarithmic search and updates under its algorithm's assumptions.

A heap solves a different problem: efficiently access an extreme-priority element. A binary min-heap requires each parent to be no greater than its children. It does not order siblings or provide efficient arbitrary-key search. An array satisfying the heap invariant is not a fully sorted array.

Choose from the query contract. An ordered asset index needing range queries suggests an ordered search structure. A pathfinder repeatedly extracting the smallest tentative cost suggests a priority queue. A structure's name is not evidence that it supports every kind of ordering efficiently. [R13: trees, heaps]

An array represents a complete binary heap

In a zero-based array, the parent of a nonroot node i is `(i-1)/2`; children are `2*i+1` and `2*i+2` when those indices exist. Check existence before access and avoid overflow in index arithmetic for unrestricted sizes. The teaching heap has a bounded input scope and uses loop conditions that avoid forming invalid children.

To push, append at the end and repeatedly exchange with the parent while the new item is smaller. To pop, remove the root, move the last element into its place, shrink the array and sift down through the smaller child. Each path has logarithmic height in a complete binary tree.

Popping a one-element heap needs special handling: after removing that element, there is no root to repair. Popping an empty heap returns absence under our contract. Duplicates are allowed and do not violate parent <= child ordering.

Tie-breaking is observable behavior

Two events can have equal priority. A queue may return them in an unspecified relative order unless the comparator defines a secondary key. For reproducible processing, use a stable sequence number or another explicit tie-breaker. The complete comparison must satisfy a strict weak ordering for standard ordering algorithms.

Do not use <= as a less-than comparator: an item would compare less than itself. For a tuple (priority, sequence), ordinary lexicographic less-than gives a clear total ordering when both components have ordinary integer order. If sequence numbers can wrap, define a bounded lifetime or a checked exhaustion policy.

Floating-point NaN complicates comparisons. A queue of finite validated costs can avoid that domain problem; otherwise specify a total-order policy rather than assuming ordinary < handles every value coherently. The pathfinding reference uses bounded nonnegative integer costs.

Test the invariant, not only the final answer

After every push and pop, scan the array and verify each child's priority is not less than its parent's. This O(n) test instrumentation is intentionally outside a claim about production push/pop complexity. It provides detailed failure localization during learning.

Use a sorted vector or multiset as a small independent oracle. Generate a deterministic sequence with duplicate and negative integer priorities, compare each extracted value and then check the heap invariant. A test that inserts random values and only checks final sorted output can reveal a defect, but checking every mutation tells you where it started.

Also explain why a min-heap cannot answer a general sorted range query as cheaply as a balanced ordered tree. The module project should document the data structure you did not choose and the requirement that would justify it.

Push 7, 2, 5, 1

After 7, the heap is [7]. Push 2 and exchange with 7: [2,7]. Push 5: [2,7,5]. Push 1 at index 3, exchange with 7 and then 2: [1,2,5,7]. This happens to look sorted, but that is not required.

Pop 1, replace it with 7, then choose the smaller child 2 and exchange: [2,7,5]. This is a valid heap and not a sorted array. The next pop is still 2. Continue and expect 5 then 7.

For equal priorities, attach insertion sequence numbers and predict their exact extraction order. Without such a rule, do not write a test that assumes stable ordering the container never promised.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_13_4.cpp`. Pseudocode excerpts are not standalone C++ source.

```
for (std::size_t child = 1; child < heap.size(); ++child) {
    const std::size_t parent = (child - 1) / 2;
    CHECK(!(heap[child] < heap[parent]));
}
```

Common failure modes

A heap is not globally sorted. A BST is not automatically balanced. A comparator using <= violates irreflexivity. An invariant scan adds testing cost; do not accidentally include it in a benchmark claiming logarithmic operation timing.

Practical assignment

Original requirement: Implement a binary min-heap with push/pop and randomized comparison to a sorted reference. Pass: validate the heap invariant after every operation and test duplicate priorities.

1. Implement a binary min-heap with explicit empty behavior and a size budget.
2. Validate parent/child order after every mutation.
3. Compare push/pop traces with a sorted independent oracle, including duplicates.

4. Add an event-priority tie-breaker and compare heap requirements with balanced-tree requirements.

Required cases: empty pop -> absent; one-element pop leaves empty; ascending and descending insertions; duplicate priorities retained; negative priorities ordered correctly; invariant after every randomized operation.

Evidence to submit: Index diagram, sift traces, invariant checker, differential tests and a data-structure decision table.

Independent transfer: A scheduler now needs all events whose times lie in an interval, not only the next event. Reconsider the structure and explain the changed query cost.

Assigned primary reading: R13: binary search trees, balanced trees and heaps.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Does a heap provide a fully sorted array?
2. What can the height of an unbalanced n-node search tree become?
3. Why define a tie-break rule in a priority queue?
4. Is [2,7,5] a valid min-heap?
5. Why is `a <= b` not a suitable less-than comparator?

Sources and provenance

Continuity with the original program

This expansion was prepared for the September28,2026 curriculum edition. The original program manifest identifies Volume03 as C++, Algorithms and Language Implementation, Modules11–15, GateC and400–650 planning hours. Its lesson IDs, titles, outcomes, prerequisites, practical assignments and assigned readings are preserved in `source/ORIGINAL_VOLUME_03_MANIFEST.json`. The 60 original quiz questions and answers were recovered from the original `CURRICULUM.html`; the expansion adds40 application questions without replacing the originals.

Teaching explanations, worked examples, code, additional exercises, reviews, bridge material and project elaborations are original instructional synthesis for this bundle. Primary reading references support the underlying concepts. No external textbook, paper, compiler distribution, proprietary SDK or font file is included. Linked resources retain their own terms. Current documentation was consulted on September28,2026; historical course archives are intentionally used for durable foundations. Not every section of every historical source was re-reviewed in full.

The original assignments are preserved even where the starter supplies only a first contract. The full lab remains a learner requirement. Complete reference examples provide comparison evidence, not pre-completed module projects. The source JSON distinguishes original questions from additions.

Primary reference directory

R06 — 6.042J Mathematics for Computer Science

MIT OpenCourseWare. Proofs, graphs, counting and discrete probability.

https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/

R11 — C++ Core Guidelines

C++ Core Guidelines editors. Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R12 — CMake tutorial

Kitware. Build systems; use documentation matching the installed release.

https://cmake.org/cmake/help/latest/guide/tutorial/index.html

R13 — 6.006 Introduction to Algorithms

MIT OpenCourseWare. Data structures, graph algorithms and dynamic programming.

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

R14 — Kaleidoscope and LLVM tutorials

LLVM Project. Compiler frontends, intermediate representation and JIT compilation.

https://llvm.org/docs/tutorial/

R15 — 6.004 Computation Structures

MIT OpenCourseWare. Logic, processor design, memory and systems.

https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/

R16 — Operating Systems: Three Easy Pieces

Remzi and Andrea Arpaci-Dusseau. Virtualization, concurrency and persistence; book by its authors.

https://pages.cs.wisc.edu/~remzi/OSTEP/

R20 — 15-466/666 Computer Game Programming

Carnegie Mellon University. Runtime systems and game-production programming.

https://graphics.cs.cmu.edu/courses/15-466/

P4-01 — CMake compile features and language standards

Kitware. Target requirements, language standards and extensions. Use the documentation matching your CMake release.

https://cmake.org/cmake/help/latest/manual/cmake-compile-features.7.html

P4-02 — What Every Computer Scientist Should Know About Floating-Point Arithmetic

David Goldberg / Oracle documentation. Author paper reproduced in official numerical-computation documentation; rounding and non-associativity background.

https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html

P4-03 — AddressSanitizer

LLVM / Clang. Compiler/runtime setup and kinds of executed memory errors detected. Detection is not an all-input proof.

https://clang.llvm.org/docs/AddressSanitizer.html

P4-04 — UndefinedBehaviorSanitizer

LLVM / Clang. Runtime instrumentation and supported checks. Platform/toolchain support must be verified.

https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

P4-05 — Language standards supported by GCC

GNU Compiler Collection. C/C++ standards and language modes. This course selects C17/C++20, not a newest-standard claim.

https://gcc.gnu.org/onlinedocs/gcc/Standards.html

P4-06 — Kaleidoscope: implementing a parser and AST

LLVM Project. Frontend architecture reading; the course language and integer semantics are independently specified and do not require LLVM libraries.

https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/LangImpl02.html

P4-07 — Fundamental types [basic.fundamental]

C++ working-draft editors / eel.is rendering. Current working-draft view, not the fixed published C++20 edition. Consult only the stable integer/representation concepts used here; later additions are outside this course baseline.

https://eel.is/c++draft/basic.fundamental

P4-08 — Vector modifiers, capacity and unordered-container requirements

C++ working-draft editors / eel.is rendering. Companion sections: https://eel.is/c++draft/vector.capacity and https://eel.is/c++draft/unord.req. Used for operation-specific invalidation rules; current draft contains later features not used by the course.

https://eel.is/c++draft/vector.modifiers

Reading by topic

Use R11 for ownership, interfaces, special members, exception safety and source-file organization; R12/P4-01 for builds; R13 for arrays, hashing, heaps, sorting, graphs and dynamic programming; R14/P4-06 for language frontends and compiler structure. R06 supports discrete proof and computability reasoning, and R15 connects representations to later hardware study. R16/R20 are cross-volume reading pointers, not dependencies of the supplied build.

Read the exact assignment printed under each lesson. Record the section actually studied and at least one exercise or derivation in the journal. The narrow course implementation deliberately differs from some reading examples: bounded integers instead of general floating-point language values, a small straight-line stack machine instead of an LLVM JIT, and mock resources instead of a graphics API. Do not merge incompatible assumptions silently.